Experiment 9

Student Name: Sejal
Branch: CSE - AIML
Semester: 4
Subject Name: DBMS

UID: 24BAI70050
Section/Group: 24AIT_KRG G1
Date of Performance: 17/4/26
Subject Code: 24CSH-298

Aim

To design and implement stored procedures in PostgreSQL for performing Create, Read, Update, and Delete (CRUD) operations on database tables in an efficient, secure, and reusable manner.

Software Requirements

- Database Management System:
  - Oracle Database Express Edition (Oracle XE)
  - PostgreSQL Database
- Database Administration Tool / Client Tool:
  - Oracle SQL Developer (for Oracle XE)
  - pgAdmin (for PostgreSQL)

Objectives

- To understand the concept and importance of stored procedures
- To implement parameterized stored procedures
- To perform CRUD operations (INSERT, SELECT, UPDATE, DELETE)
- To improve database performance and security
- To gain industry-relevant procedural SQL experience.

Procedure

1. Initialization of Environment
   The PostgreSQL environment was started using pgAdmin or terminal. A connection was established with the required database to perform the experiment.
2. Creation of Table
   A table named *student* was created with appropriate attributes such as ID, name, age, and course. The ID field was defined as the primary key to uniquely identify each record.
3. Implementation of INSERT Procedure
   A stored procedure was designed to insert new records into the student table. The procedure accepts parameters such as name, age, and course, and adds a new entry into the table. This demonstrates the Create operation of CRUD.
4. Execution of INSERT Procedure
   The insert procedure was executed by passing appropriate values as arguments. The successful addition of records confirmed the working of the procedure.
5. Implementation of READ Operation
   A function was created to retrieve data from the student table. It returns all the records present in the table, allowing users to view stored data efficiently.
6. Execution of READ Operation
   The function was executed to display all the records from the table. The output confirmed that data retrieval was functioning correctly.
7. Implementation of UPDATE Procedure
   A stored procedure was created to update existing records in the table. The procedure takes parameters such as ID and updated values, and modifies the corresponding record.
8. Execution of UPDATE Procedure
   The update procedure was executed to modify specific records. The changes were verified by retrieving the updated data.
9. Implementation of DELETE Procedure
   A stored procedure was developed to delete records from the table based on a given ID. This ensures controlled removal of data.

10. Execution of DELETE Procedure
    The delete procedure was executed to remove selected records. The deletion was verified by checking the remaining data in the table.
11. Verification of All Operations
    All CRUD operations were verified by retrieving and observing the data after each operation to ensure correctness and consistency.
12. Testing and Validation
    Multiple test cases were performed by inserting, updating, and deleting different records to validate the robustness and correctness of the stored procedures.

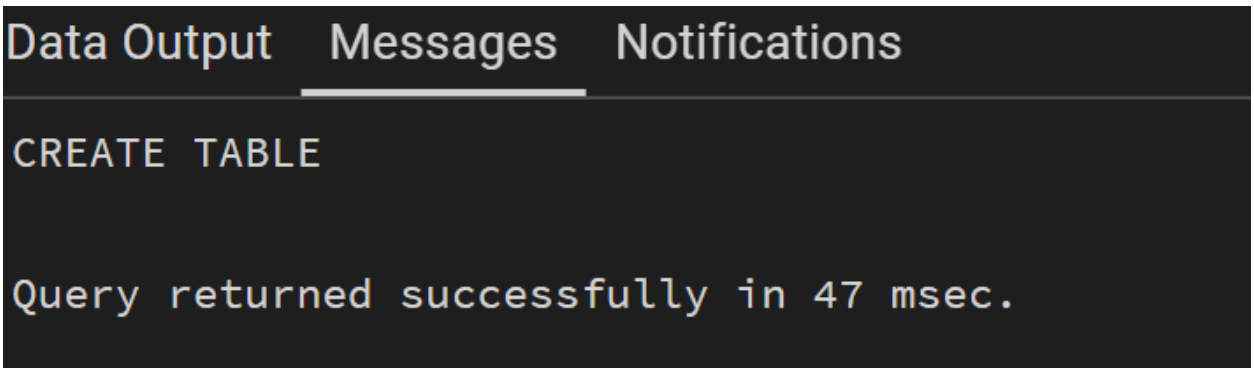
Input/Output Analysis

SQL Input Queries

Step 1: Create Table

```
CREATE TABLE student (
   id SERIAL PRIMARY KEY,
   name VARCHAR(50),
   age INT,
   course VARCHAR(50)
);
```

Output



Step 2: CREATE (Insert Procedure)

```
CREATE OR REPLACE PROCEDURE insert_student(
   s_name VARCHAR,
   s_age INT,
   s_course VARCHAR
)
LANGUAGE plpgsql
AS $$
BEGIN
   INSERT INTO student(name, age, course)
   VALUES (s_name, s_age, s_course);
END;
$$;
```

Output

Data Output Messages Notifications

```
CREATE PROCEDURE

Query returned successfully in 46 msec.
```

Step 3: READ (Fetch Data Procedure)

```
CREATE OR REPLACE PROCEDURE get_students()
LANGUAGE plpgsql
AS $$
BEGIN
   PERFORM * FROM student;
END;
$$;
```

Output

Data Output Messages Notifications

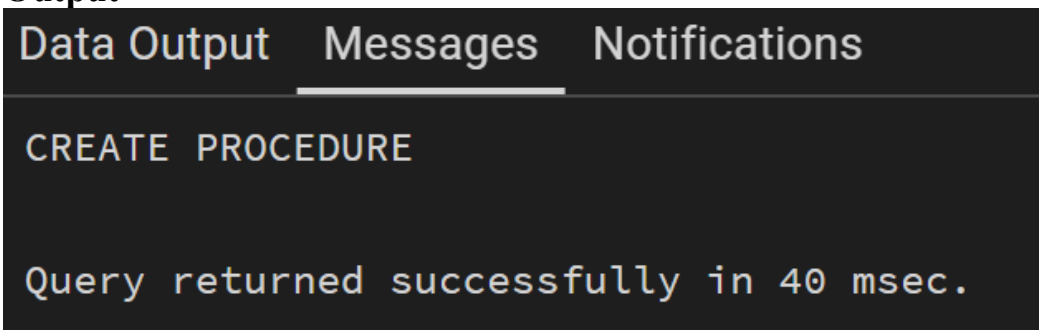
```
CREATE PROCEDURE

Query returned successfully in 38 msec.
```

Step 4: UPDATE Procedure

```
CREATE OR REPLACE PROCEDURE update_student(
   s_id INT,
   s_name VARCHAR,
   s_age INT,
   s_course VARCHAR
)
LANGUAGE plpgsql
AS $$
BEGIN
   UPDATE student
   SET name = s_name,
      age = s_age,
      course = s_course
   WHERE id = s_id;
END;
$$;
```

Output



Step 5: DELETE Procedure

```
CREATE OR REPLACE PROCEDURE delete_student(s_id INT)
LANGUAGE plpgsql
AS $$
BEGIN
   DELETE FROM student WHERE id = s_id;
END;
$$;
```

Output

```
Data Output   Messages   Notifications

CREATE PROCEDURE

Query returned successfully in 38 msec.
```

Step 6: Calling Procedures

CALL insert_student('Sejal', 20, 'CSE');
CALL update_student(1, 'Sejal Patial', 21, 'AI-ML');
CALL delete_student(1);

```
Data Output   Messages   Notifications

CALL

Query returned successfully in 37 msec.
```

Learning Outcomes

- Gained a clear understanding of stored procedures and their role in database management systems.
- Learned how to design and implement parameterized stored procedures for performing various database operations.
- Acquired practical knowledge of CRUD operations (Create, Read, Update, Delete) using PostgreSQL.
- Understood how stored procedures improve performance by reducing query execution time and network overhead.
- Developed the ability to enhance database security by restricting direct access to tables through procedures.